Two-Level Autoresearch: Optimizing LLM Policy Synthesis in Sequential Social Dilemmas

Víctor Gallego
victor.gallego@komorebi.ai
Komorebi AI Technologies
Madrid, Spain

Abstract

We demonstrate an autonomous AI agent that discovers effective configurations for a complex multi-agent system—without human intervention. A *researcher agent* $\mathcal{R}$ (Claude Opus 4.6) iteratively modifies the synthesis pipeline—prompts, feedback, helpers, iteration logic—under which a *policy-synthesizer* LLM $\mathcal{M}$ generates cooperative strategies for Sequential Social Dilemmas, following the *autoresearch* paradigm. Across 12 autonomous runs (two games, two policy LLMs, two welfare objectives), the researcher reliably exceeds baselines: in Cleanup, efficiency improves by 66–263% while tightening run-to-run spread to near-zero. Maximin-optimized pipelines achieve near-perfect equality (E=0.98) with negligible efficiency loss, discovering duty rotation as the key fairness mechanism—never found under efficiency optimization. The full-pipeline approach outperforms prompt-only optimization (GEPA) by 2–3×. The contrast between Cleanup and Gathering reveals that game structure determines whether fairness requires explicit optimization: provision dilemmas need designed mechanisms, while restraint dilemmas achieve fairness as a byproduct of coordination.

CCS Concepts

• **Computing methodologies → Multi-agent systems**; **Cooperation and coordination**; *Machine learning*.

Keywords

sequential social dilemmas, LLM policy synthesis, multi-agent systems, automated research, mechanism design

ACM Reference Format:
Víctor Gallego. 2026. Two-Level Autoresearch: Optimizing LLM Policy Synthesis in Sequential Social Dilemmas. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Background

We build on the iterative LLM policy synthesis framework of Gallego [3]. An SSD is a partially observable Markov game $\mathcal{G} = \langle N, \mathcal{S}, \{A_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$. Social outcomes are evaluated via four metrics [8]: efficiency U, equality E, sustainability S, and peace P. We additionally consider the *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which maximizes the return of the worst-off agent.

We study two SSDs that represent complementary dilemma types. **Cleanup** [4] is a *public goods provision* game (N=10): a river accumulates waste while an orchard grows apples. Cleaning costs −1 per beam but enables apple regrowth for everyone (+1 per apple). The dilemma: free-ride on cleaning vs. contribute. **Gathering** [7, 8] is a *common pool resource* game (N=4): agents collect shared apples and may tag rivals to temporarily remove them. The dilemma: over-harvest and aggress vs. cooperate and share. The games differ in their cost structure—asymmetric provision vs. symmetric restraint—a distinction that drives our key findings.

A frozen LLM $\mathcal{M}$ acts as a policy synthesizer. Given a system prompt p and a feedback prompt q_k, it generates a Python policy $\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k^\ell))$, where ℓ specifies the feedback level and $\mathcal{F}_k = (\bar{r}_k, \mathbf{m}_k)$ contains the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$. All N agents execute the same policy (homogeneous self-play). Gallego [3] compared two hand-designed feedback levels—sparse (reward-only) and dense (reward + social metrics)—and showed that dense feedback consistently improves cooperative outcomes. The dense feedback configuration achieves U=1.93 (mean) / 2.70 (max) with Gemini and U=0.86 / 1.56 with Sonnet on Cleanup (4 runs each).

2 Two-Level Framework

We introduce a two-level system where a *researcher agent* $\mathcal{R}$ autonomously discovers configurations that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates them, and iterates can serve as the inner loop. The key insight is that the entire inner-loop codebase—not just the feedback level—is a designable artifact that an AI agent can search over. Figure 1 illustrates the architecture.

2.1 Configuration Space

Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-loop setup:

$$c = (p,\ \ell,\ \phi,\ \mathcal{H},\ \iota,\ \nu) \tag{1}$$

where p is the system prompt, ℓ is the feedback template, ϕ is the feedback construction function, $\mathcal{H}$ is the helper function library, ι specifies the iteration logic (number of iterations K, sampling strategy, temperature schedule), and ν is the validation pipeline. Table 1 provides concrete examples.

The sparse vs. dense comparison of [3] corresponds to a single binary choice within the ℓ component. Our framework opens the full configuration space to automated search.

2.2 Inner Loop (Policy Synthesis)

Given a configuration c, the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \textsc{InnerLoop}(\mathcal{M}, \mathcal{G}, c) \tag{2}$$

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

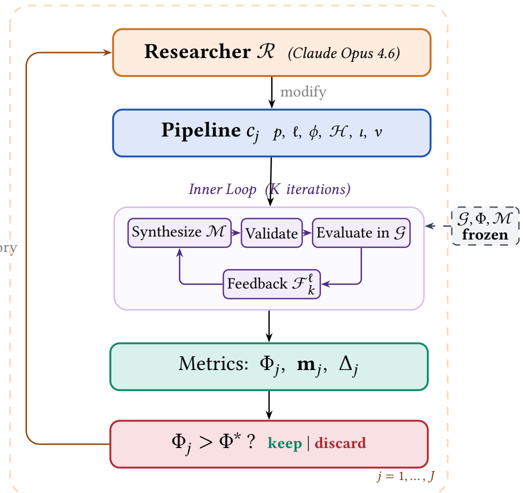

Figure 1: Two-level automated research framework (Algorithm 1). *Outer loop*: the researcher $\mathcal{R}$ modifies the pipeline configuration c_j, runs the inner loop, and keeps or discards based on Φ. *Inner loop*: the policy LLM $\mathcal{M}$ iteratively synthesizes, validates, and evaluates policies in the frozen environment $\mathcal{G}$.

Table 1: Configuration components modifiable by the researcher. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

	Scope	Examples
p	System prompt	Hints, worked examples
ℓ	Feedback template	Metric subset, framing
ϕ	Feedback fn	Derived metrics, trends
$\mathcal{H}$	Helper library	Pathfinding, zones
ι	Iteration logic	K, eval seeds, best-of-n
ν	Validation	Safety checks, smoke tests

Each iteration k proceeds in four stages following [3]: (1) **Synthesize**: $\mathcal{M}$ receives prompt p, previous policy π_{k-1}, and feedback $\mathcal{F}_{k-1}^\ell$, generating a new policy π_k with access to helpers $\mathcal{H}$; (2) **Validate**: AST-based safety checks and smoke tests, with regeneration on failure; (3) **Evaluate**: self-play over $|S|$ seeds yielding reward $\bar{r}_k$ and metrics $\mathbf{m}_k$; (4) **Feedback**: ϕ constructs the next-iteration prompt from $(\pi_k, \bar{r}_k, \mathbf{m}_k)$ per template ℓ.

The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \textsc{Eval}(\pi_c^*;\ \mathcal{G},\ S_{\text{held-out}}) \tag{3}$$

where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \qquad \text{(utilitarian efficiency)} \tag{4}$$

$$\Phi_{\min} = \min_i R_i \qquad \text{(Rawlsian maximin)} \tag{5}$$

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt for researcher $p_{\mathcal{R}}$, initial configuration c_0, outer iterations J, ground-truth objective Φ, held-out seeds $S_{\text{held-out}}$
Ensure: Best configuration c^*, best policy π^*

```
π*_0 ← InnerLoop(M, G, c_0)
Φ_0 ← Eval(π*_0; G, S_held-out)
history ← {(c_0, Φ_0, m_0, ∅)}
for j = 1, ..., J do
    c_j ← R(p_R, code(c_{j−1}), history)
    if ¬ValidateConfig(c_j) then retry ≤ R times
    π*_j ← InnerLoop(M, G, c_j)
    Φ_j ← Eval(π*_j; G, S_held-out)
    Δ_j ← Diff(c_{j−1}, c_j)                // code diff
    history ← history ∪ {(c_j, Φ_j, m_j, Δ_j)}
end for
j* ← arg max_j Φ_j
return c_{j*}, π*_{j*}
```

2.3 Outer Loop (Automated Research)

The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch paradigm [6], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes, observing outcomes, and refining. The procedure is formalized in Algorithm 1.

At each iteration, $\mathcal{R}$ receives the full source code of c_{j-1}, the experiment history (diffs Δ_i, scores Φ_i, metrics $\mathbf{m}_i$), and the environment source (read-only). Concretely, $\mathcal{R}$ is a coding agent (Claude Code CLI) that edits Python source files, runs evaluations, and commits changes to a git branch—the same workflow a human researcher would follow.

2.4 Design Principles

Four principles constrain the framework. *Separation of concerns*: $\mathcal{R}$ and $\mathcal{M}$ are distinct LLM calls—$\mathcal{R}$ reasons about what information helps $\mathcal{M}$; $\mathcal{M}$ reasons about what policy to write. *Read-only environment*: $\mathcal{R}$ can read but not modify the simulator, preventing trivial solutions. *Fixed compute budget*: each inner loop runs exactly K iterations with $|S|$ seeds; the researcher changes what happens *within* those iterations, not how many there are. *Diff-based history*: $\mathcal{R}$ sees code diffs Δ_j alongside outcomes, enabling causal reasoning about which modifications drove improvements.

3 Connection to Automated Mechanism Design

The two-level structure admits a mechanism design interpretation [2]: the researcher $\mathcal{R}$ acts as a *principal* that controls the information structure, action space (helper functions), and incentive framing under which $\mathcal{M}$ operates; the configuration c is the *mechanism*; and the outcome is the social welfare of π_c^*. Since $\mathcal{M}$ is not strategically self-interested but has bounded rationality (its effectiveness depends on the tools and information provided), the researcher's task is closer to *information design* [5]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments show that the researcher designs qualitatively different information structures depending on Φ.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase: $\mathcal{R}$ edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates—without human intervention.

Design. For Cleanup: 2 policy LLMs × 2 objectives × 2 replications = 8 runs. For Gathering: 2 policy LLMs × 1 objective × 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game's symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [3]. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework's behavior when the policy synthesizer has substantially lower capability (Appendix C).

Baselines. The hand-designed dense feedback (reward + social metrics) configuration from [3] serves as the initial pipeline c_0 for all runs. On Cleanup (N=10), this baseline achieves U=1.93/2.70 (mean/max) with Gemini and U=0.86/1.56 with Sonnet across 4 runs each. We additionally compare against GEPA [1], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt p, whereas our researcher modifies the full pipeline $(p, \ell, \phi, \mathcal{H}, \iota, \nu$ We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method (each GEPA step costs ~3 policy evaluations, comparable to our inner loop's K+1=4).

Inner loop parameters. K=3 synthesis iterations (or K=2 if the researcher elects), evaluated over $|S|$=5–12 random seeds (researcher-configurable). The researcher ran J=3–17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 2 presents the main results, comparing autoresearch against the hand-designed baseline of [3] and GEPA prompt optimization [1].

Finding 1: The researcher reliably improves over baselines and outperforms prompt-only optimization. All 12 runs show substantial improvement (Figure 2). In Cleanup, the researcher exceeds the baseline by 66% with Gemini (U: 1.93 → 3.20) and 263% with Sonnet (U: 0.86 → 3.12), nearly closing the inter-model gap despite a 2× baseline disadvantage. Results are remarkably consistent: Gemini's runs cluster within U=3.20–3.25 (gap 0.05) and Sonnet's within 3.12–3.14 (gap 0.02), despite baselines spanning 0.29–2.70. In Gathering, all four runs converge to U≈2.5. Compared to GEPA, autoresearch achieves 2× higher efficiency in Cleanup (3.20 vs. 1.60), 2× higher maximin (290 vs. 143), and 20% higher efficiency in Gathering (2.49 vs. 2.08), with far less run-to-run variance—highlighting the advantage of modifying the full pipeline over prompt-only search.

Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini). Maximin-optimized Gemini pipelines sacrifice only 1% efficiency (U: 3.16 vs. 3.20) while achieving near-perfect equality (E: 0.98 vs. 0.55) and transforming maximin from −99 to +290. The key mechanism is *fair duty rotation*—time-based cycling of cleaning responsibilities—which simultaneously improves worst-off welfare and collective output. For Sonnet, a moderate tradeoff exists (U:

Table 2: Results. Mean / max across 2 runs per condition (4 for Cleanup baselines). Baseline: unmodified pipeline with dense feedback from [3]. GEPA: automated prompt optimization [1] with matched compute budget.

Policy LLM	Target	U	E	$\min_i R_i$
Cleanup (N=10) — Baseline				
Gemini 3.1 Pro	—	1.93/2.70	0.17/0.62	−159/−84
Sonnet 4.6	—	0.86/1.56	−0.02/0.25	−151/−59
Cleanup (N=10) — GEPA [1]				
Gemini 3.1 Pro	Φ_U	1.60/2.16	−0.01/0.62	—
	$\Phi_{\min}$	1.76/2.76	0.90/0.96	143/245
Cleanup (N=10) — Autoresearch				
Gemini 3.1 Pro	Φ_U	**3.20**/3.25	0.55/0.61	−211/−182
	$\Phi_{\min}$	3.16/3.19	**0.98**/0.98	**290**/296
Sonnet 4.6	Φ_U	**3.12**/3.14	0.66/0.70	−196/−146
	$\Phi_{\min}$	2.57/2.93	**0.91**/0.97	**179**/200
Gathering (N=4) — Baseline				
Gemini 3.1 Pro	—	2.04/2.42	0.90/0.98	412/571
Sonnet 4.6	—	0.03/0.03	0.54/0.54	0/0
Gathering (N=4) — GEPA [1]				
Gemini 3.1 Pro	Φ_U	2.08/2.35	0.94/0.96	—
Gathering (N=4) — Autoresearch				
Gemini 3.1 Pro	Φ_U	2.49/2.51	0.98/0.98	582/582
Sonnet 4.6	Φ_U	2.52/2.52	0.96/0.98	576/597

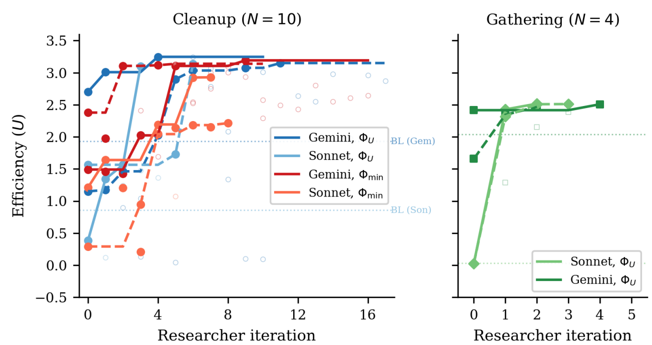

Figure 2: Efficiency (U) across researcher iterations. *Left*: Cleanup (N=10), 8 runs (2 LLMs × 2 objectives). Solid = kept, open = discarded. Dashed = baseline [3]. *Right*: Gathering (N=4), 4 runs. All converge despite diverse starting points.

3.12 → 2.57), reflecting difficulty implementing complex coordination from strategic hints alone.

Finding 3: Game structure determines whether fairness requires explicit optimization. In Cleanup (asymmetric costs), maximin optimization is required to reach $E > 0.9$. In Gathering (symmetric costs), efficiency optimization alone achieves $E > 0.94$. This generalizes: *provision* dilemmas require designed fairness mechanisms, while *restraint* dilemmas achieve fairness as a byproduct of coordination. The researcher independently discovers this, creating role differentiation only for Cleanup.

Finding 4: Convergent discovery of qualitatively different strategies per objective. Despite independent runs, the researcher converges on the same core strategies within each condition (Table 4, Appendix A). In Cleanup, waste-counting helpers and zone partitioning appear across all 8 runs; the key difference is *role rotation* (4/4 maximin runs, 0/4 efficiency runs). In Gathering, the optimization is purely spatial (BFS-based Voronoi partitioning) and temporal (respawn-aware positioning).

Failure modes. Common discarded-experiment patterns: over-prescription (too many hints cause generation failures), iteration regression ($K \geq 4$ causes catastrophic over-refinement), feedback overload (per-agent diagnostics cause over-correction), and variance exposure ($U=3.2$ then 0.09 on different seeds).

5 Discussion and Conclusion

We have presented a two-level framework where an AI coding agent autonomously discovers pipeline configurations for multi-agent policy synthesis. Across 12 runs, the researcher reliably exceeds baselines and converges on qualitatively similar strategies per condition, requiring no domain-specific scaffolding. Our results support the mechanism design interpretation: under Φ_U, the researcher reveals efficiency-oriented information guiding $\mathcal{M}$ toward productive role allocation; under $\Phi_{\min}$, it reveals fairness-oriented structure (rotation schedules, equity feedback) guiding egalitarian coordination. The Cleanup–Gathering contrast rediscovers a known structural insight—provision dilemmas require designed fairness mechanisms while restraint dilemmas achieve fairness naturally—suggesting this is a robust feature of these environments.

Oversight. Because $\mathcal{R}$ operates on a git repository, the full history of diffs and outcomes is auditable. The keep/discard decision is a natural intervention point for human oversight, and the separation between $\mathcal{R}$ and the frozen Φ creates a delegation boundary: the human defines *what* to optimize, the agent decides *how*.

Limitations. We use only 2 replications per condition and two small gridworld environments. The number of outer iterations varies (3–17), making cross-condition comparison approximate.

Future work. Key directions include: new domains (applying the framework to other LLM pipelines), component ablation, adversarial objectives ($\Phi = R_0$, connecting to reward hacking [3]), cross-game transfer, and heterogeneous policies. Code: https://github.com/vicgalle/llm-policies-social-dilemmas.

References

[1] Lakshya A Agrawal et al. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457* (2026).

[2] Vincent Conitzer and Tuomas Sandholm. 2002. Complexity of Mechanism Design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*. 103–110.

[3] Víctor Gallego. 2026. Cooperation and Exploitation in LLM Policy Synthesis for Sequential Social Dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*.

[4] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. 2018. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*.

[5] Emir Kamenica and Matthew Gentzkow. 2011. Bayesian Persuasion. *American Economic Review* 101, 6 (2011), 2590–2615.

[6] Andrej Karpathy. 2026. autoresearch: AI agents running research on single-GPU nanochat training automatically. https://github.com/karpathy/autoresearch.

[7] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. 2017. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*. 464–473.

[8] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. 2017. A multi-agent reinforcement learning model of common-pool resource appropriation. In *Advances in Neural Information Processing Systems*, Vol. 30.

A Additional Results

Table 3: Structural analogy between autoresearch [6] for LLM pretraining and our framework for multi-agent policy synthesis.

	Autoresearch	This work
Modifiable artifact	`train.py` (model + optimizer)	Inner-loop codebase $(p, \ell, \phi, \mathcal{H}, \iota, \nu)$
Frozen infrastructure	`prepare.py` (data, tokenizer, eval)	Environment $\mathcal{G}$, eval Φ, LLM $\mathcal{M}$
Metric	val_bpb ↓	Φ (social welfare) ↑
Budget	5 min wall clock	K iters × $\lvert S \rvert$ seeds
Agent	AI coding agent	Researcher $\mathcal{R}$

Table 4: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	**0/4**	**4/4**

B Compute Requirements

Table 5 reports wall-clock time for all 12 autonomous runs and the 6 GEPA comparison runs. *Inner loop* measures total time spent on policy LLM generation and environment simulation across all evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher agent's analysis, code editing, and decision-making between evaluations.

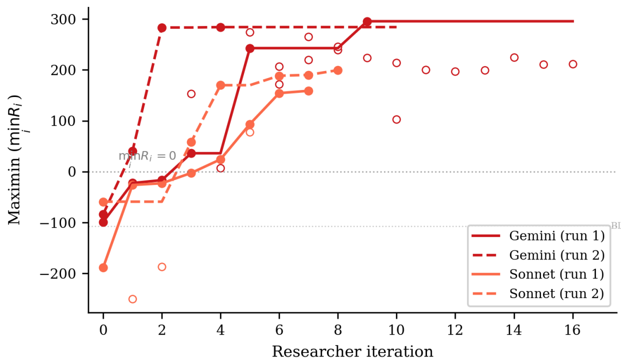

Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ∼290 while Sonnet reaches ∼160–200. The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

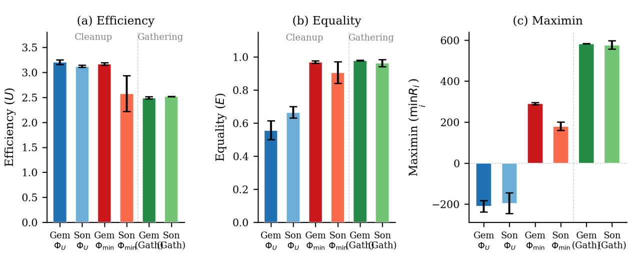

Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to $U \approx 2.5$–3.2. (b) Equality: maximin-optimized Cleanup runs achieve $E \approx 1.0$, while efficiency-optimized runs show $E \approx 0.5$–0.7; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ($\min_i R_i \approx -200$), while maximin optimization transforms it to +290 (Gemini) or +179 (Sonnet). Gathering achieves high maximin (∼580) under efficiency optimization alone. Error bars show s.d. across runs.

Cost breakdown. Each inner loop evaluation involves K=2–3 policy LLM generation calls (each ∼2k input tokens, ∼1.5k output tokens) followed by multi-seed simulation (5–12 seeds, ∼15–30s total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking ∼2× longer than Gemini due to extended thinking. The researcher agent (Claude Opus 4.6, running via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary terms, the researcher agent is the dominant cost: each outer iteration consumes ∼50–100k context tokens in the Opus session, whereas each inner loop evaluation uses only ∼5–10k policy LLM tokens. The 6 GEPA comparison runs required 5.0h of compute with no researcher agent, operating at lower cost per run but achieving substantially lower performance (Table 2).

Table 5: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

Policy LLM	Φ	Evals	Inner loop (h)	Per eval (min)	Total (h)
Cleanup (N=10)					
Gemini	Φ_U	11	2.6	14	3.7
Gemini	Φ_U	18	4.1	14	6.4
Sonnet	Φ_U	4	2.1	32	6.1
Sonnet	Φ_U	7	5.0	43	6.1
Gemini	$\Phi_{\min}$	17	3.0	11	4.3
Gemini	$\Phi_{\min}$	11	3.6	20	5.0
Sonnet	$\Phi_{\min}$	8	4.5	33	8.8
Sonnet	$\Phi_{\min}$	9	5.3	36	13.2
Gathering (N=4)					
Sonnet	Φ_U	3	1.7	33	2.2
Gemini	Φ_U	5	1.4	17	1.9
Gemini	Φ_U	3	0.9	19	1.1
Sonnet	Φ_U	4	2.3	35	3.4
All 12 runs		100	36.6	22	62.2
GEPA (6 runs)		6	5.0	50	—

C Smaller Open-Weight Model: Gemma 4 26B

We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model substantially smaller than the frontier models in the main experiments. We run three experiments—two on Cleanup (N=10) optimizing efficiency and maximin respectively, and one on Gathering (N=4) optimizing efficiency—all with Opus 4.6 as the researcher $\mathcal{R}$.

Setup. In all three experiments, Gemma starts from complete failure: the baseline pipeline produces U=−10.0 on Cleanup (agents CLEAN-spam without collecting apples) and U=0.0 on Gathering (broken BFS calls), compared to U=1.93/0.86 (Gemini/Sonnet on Cleanup) and U=2.04/0.03 (Gemini/Sonnet on Gathering). The researcher ran J=10–18 outer iterations per condition.

Results. Table 6 presents the best configurations discovered. Under efficiency optimization, the researcher achieves U=0.87—a dramatic recovery from the broken baseline but far below frontier models (U≈3.2). The researcher compensates for the model's limited capability by reducing inner-loop iterations to K=1 (avoiding the regression that plagued K≥2 runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Maximin optimization rescues efficiency. Strikingly, under maximin optimization the researcher achieves *higher* efficiency (U=1.71) than under direct efficiency optimization (U=0.87), alongside strong equality (E=0.94) and positive worst-off welfare ($\min_i R_i$=137). This reversal—absent in frontier models, where both objectives yield U≈3.1–3.2—occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher

Table 6: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: dense feedback pipeline from [3].

Game	Target	U	E	$\min_i R_i$	Keep rate
Cleanup	Baseline	−10.0	1.00†	−1000	—
	Φ_U	0.87	−1.11	−371	6/19
	$\Phi_{\min}$	**1.71**	**0.94**	**137**	9/17
Gathering	Baseline	0.0	1.00†	0	—
	Φ_U	**2.44**	**0.98**	**580**	4/11

† Equality is trivially 1.0 because all agents receive near-zero reward.

converges on a local optimum—static role assignment with 25% dedicated cleaners—that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization.* The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model's difficulty implementing complex strategies from strategic hints alone.

Gathering: full recovery on a simpler game. On Gathering (N=4), the researcher brings Gemma from U=0.0 to U=2.44 with E=0.98 and $\min_i R_i$=580, after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini: U=2.49, Sonnet: U=2.52; Table 2). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found in the main experiments, and increases inner-loop iterations to K=5 to allow sufficient refinement. The contrast with Cleanup—where Gemma peaks at U=0.87/1.71 versus frontier $U\approx$3.2—suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).